

BAB 7

RELASI ANTAR OBJECT: ASSOCIATION, AGGREGATION, DAN COMPOSITION

7.1 Pendahuluan

Pada bab sebelumnya, mahasiswa telah mempelajari **abstract class** dan **interface**. Materi tersebut membantu mahasiswa memahami bagaimana membuat rancangan program yang lebih fleksibel dan terstruktur.

Pada BAB 7 ini, mahasiswa akan mempelajari **relasi antar object**. Dalam program yang lebih besar, object biasanya tidak berdiri sendiri. Object saling terhubung dan bekerja sama untuk menyelesaikan suatu proses.

Sebagai contoh, dalam sistem akademik, object `Mahasiswa` dapat berhubungan dengan object `MataKuliah`. Object `Dosen` dapat berhubungan dengan object `KelasPraktikum`. Object `Gedung` dapat memiliki beberapa object `RuangKelas`.

Relasi antar object penting dipahami karena OOP tidak hanya membahas cara membuat class, tetapi juga bagaimana class saling berhubungan. Dengan memahami relasi ini, mahasiswa dapat membuat desain program yang lebih dekat dengan sistem nyata.

7.2 Tujuan Pembelajaran

Setelah menyelesaikan praktikum pada bab ini, mahasiswa diharapkan mampu:

1. Menjelaskan pengertian relasi antar object.
2. Membedakan association, aggregation, dan composition.
3. Membuat relasi antar class dalam Java.
4. Menggunakan object sebagai atribut pada class lain.
5. Membuat program sederhana yang melibatkan beberapa class.
6. Menentukan jenis relasi yang tepat dalam sebuah studi kasus.

7.3 Pengertian Relasi Antar Object

Relasi antar object adalah hubungan antara dua atau lebih object dalam sebuah program. Hubungan ini terjadi karena satu object membutuhkan data atau fungsi dari object lain.

Dalam kehidupan nyata, banyak hal yang saling berhubungan. Mahasiswa berhubungan dengan mata kuliah, dosen berhubungan dengan kelas, dan gedung berhubungan dengan ruang kelas. Hubungan seperti ini dapat dimodelkan dalam program menggunakan relasi antar class.

Contoh relasi antar object:

- Mahasiswa **mengambil** MataKuliah
- Dosen **mengajar** MataKuliah
- KelasPraktikum **memiliki daftar** Mahasiswa

- Gedung memiliki beberapa RuangKelas

Relasi antar object biasanya digunakan untuk menggambarkan hubungan **uses-a** atau **has-a**. Hubungan **uses-a** berarti suatu object menggunakan object lain, sedangkan **has-a** berarti suatu object memiliki object lain.

7.4 Association

Association adalah relasi umum antara dua class atau object. Pada relasi ini, satu object dapat mengenal, menggunakan, atau berhubungan dengan object lain, tetapi tidak memilikinya secara kuat.

Contoh association adalah hubungan antara Mahasiswa dan MataKuliah. Seorang mahasiswa dapat mengambil sebuah mata kuliah, tetapi mata kuliah tetap dapat ada tanpa mahasiswa tersebut. Begitu juga sebaliknya, mahasiswa tetap ada walaupun tidak sedang mengambil mata kuliah tertentu.

Dalam association, kedua object biasanya masih dapat berdiri sendiri. Relasinya hanya menunjukkan bahwa keduanya saling berhubungan dalam suatu proses.

Contoh Association

```
class MataKuliah {
    String namaMataKuliah;

    MataKuliah(String namaMataKuliah) {
        this.namaMataKuliah = namaMataKuliah;
    }

    void tampilkanMataKuliah() {
        System.out.println("Mata Kuliah: " + namaMataKuliah);
    }
}
```

```
class Mahasiswa {
    String nama;

    Mahasiswa(String nama) {
        this.nama = nama;
    }

    void ambilMataKuliah(MataKuliah mk) {
        System.out.println(nama + " mengambil mata kuliah " +
mk.namaMataKuliah);
    }
}
```

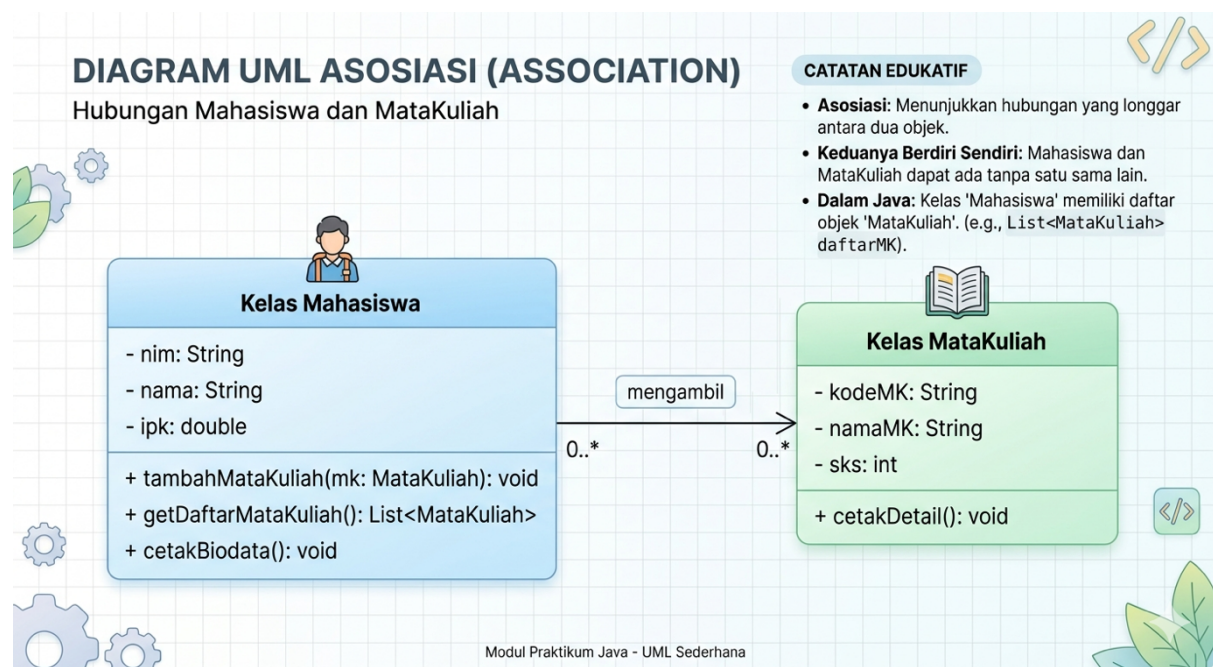
```
public class DemoAssociation {
    public static void main(String[] args) {
        Mahasiswa mhs = new Mahasiswa("Andi");
        MataKuliah mk = new MataKuliah("Pemrograman Berorientasi
Objek");

        mhs.ambilMataKuliah(mk);
    }
}
```

Output:

Andi mengambil mata kuliah Pemrograman Berorientasi Objek

Pada contoh tersebut, object Mahasiswa menggunakan object MataKuliah melalui parameter method `ambilMataKuliah()`. Hubungan ini termasuk association karena Mahasiswa dan MataKuliah tetap dapat berdiri sendiri.



Gambar 7.1 Ilustrasi Association antara Mahasiswa dan MataKuliah

Gambar ini menunjukkan hubungan association antara class **Mahasiswa** dan **MataKuliah**. Object mahasiswa dapat menggunakan object mata kuliah, tetapi keduanya tetap dapat berdiri sendiri tanpa ketergantungan kepemilikan yang kuat.

7.5 Aggregation

Aggregation adalah relasi yang menunjukkan hubungan **memiliki**, tetapi object yang dimiliki masih dapat berdiri sendiri. Relasi ini lebih kuat daripada association, tetapi tidak sekuat composition.

Contoh aggregation adalah hubungan antara `KelasPraktikum` dan `Mahasiswa`. Sebuah kelas praktikum dapat memiliki beberapa mahasiswa. Namun, jika kelas praktikum dihapus, object mahasiswa tetap dapat ada.

Dalam UML, aggregation biasanya digambarkan dengan simbol **diamond kosong** pada sisi class pemilik.

Contoh Aggregation

```
class Mahasiswa {
    String nama;

    Mahasiswa(String nama) {
        this.nama = nama;
    }

    void tampilkanNama() {
        System.out.println("Mahasiswa: " + nama);
    }
}
```

```
class KelasPraktikum {
    String namaKelas;
    Mahasiswa[] daftarMahasiswa;

    KelasPraktikum(String namaKelas, Mahasiswa[] daftarMahasiswa) {
        this.namaKelas = namaKelas;
        this.daftarMahasiswa = daftarMahasiswa;
    }

    void tampilkanAnggotaKelas() {
        System.out.println("Kelas Praktikum: " + namaKelas);
        System.out.println("Daftar Mahasiswa:");

        for (Mahasiswa mhs : daftarMahasiswa) {
            mhs.tampilkanNama();
        }
    }
}
```

```
public class DemoAggregation {
    public static void main(String[] args) {
        Mahasiswa m1 = new Mahasiswa("Andi");
        Mahasiswa m2 = new Mahasiswa("Budi");
        Mahasiswa m3 = new Mahasiswa("Citra");

        Mahasiswa[] daftar = {m1, m2, m3};

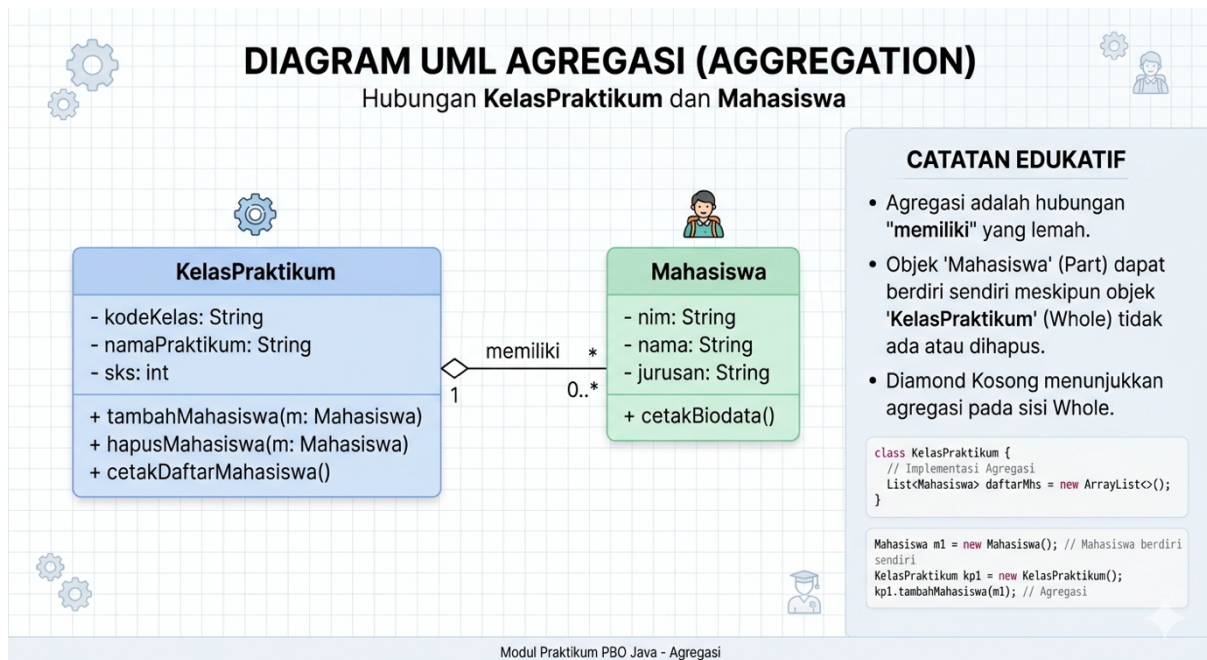
        KelasPraktikum kelas = new KelasPraktikum("PBO IF-01", daftar);
        kelas.tampilkanAnggotaKelas();
    }
}
```

Output:

```
Kelas Praktikum: PBO IF-01
Daftar Mahasiswa:
```

Mahasiswa: Andi
 Mahasiswa: Budi
 Mahasiswa: Citra

Pada contoh tersebut, object `Mahasiswa` dibuat terlebih dahulu di luar class `KelasPraktikum`. Setelah itu, object-object tersebut dimasukkan ke dalam object `KelasPraktikum`. Ini menunjukkan aggregation karena mahasiswa tetap dapat ada meskipun object kelas praktikum tidak digunakan lagi.



Gambar 7.2 Ilustrasi Aggregation antara `KelasPraktikum` dan `Mahasiswa`

Gambar ini menunjukkan hubungan aggregation, yaitu class `KelasPraktikum` memiliki beberapa object `Mahasiswa`. Namun, object `Mahasiswa` tetap dapat berdiri sendiri meskipun object `KelasPraktikum` tidak ada.

7.6 Composition

Composition adalah relasi kepemilikan yang kuat antara object utama dan object bagian. Pada relasi ini, object bagian sangat bergantung pada object utama.

Contoh composition adalah hubungan antara `Gedung` dan `RuangKelas`. Sebuah gedung memiliki ruang kelas. Dalam konteks program, ruang kelas dapat dianggap sebagai bagian dari gedung. Jika object gedung tidak dibuat, maka object ruang kelas sebagai bagian dari gedung juga tidak dibuat.

Dalam UML, composition biasanya digambarkan dengan simbol **diamond hitam** pada sisi class pemilik.

Contoh Composition

```
class RuangKelas {
    String namaRuang;

    RuangKelas(String namaRuang) {
        this.namaRuang = namaRuang;
    }

    void tampilkanRuang() {
        System.out.println("Ruang Kelas: " + namaRuang);
    }
}
```

```
class Gedung {
    String namaGedung;
    RuangKelas ruang1;
    RuangKelas ruang2;

    Gedung(String namaGedung) {
        this.namaGedung = namaGedung;

        // Object RuangKelas dibuat di dalam class Gedung
        ruang1 = new RuangKelas("Ruang 101");
        ruang2 = new RuangKelas("Ruang 102");
    }

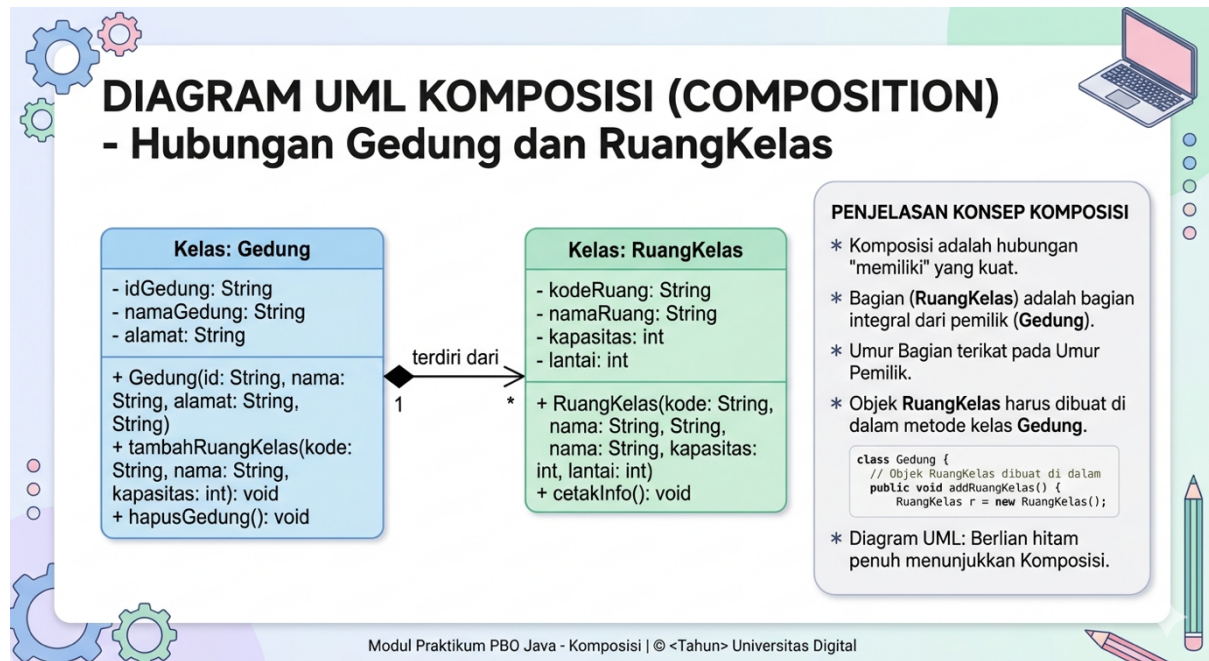
    void tampilkanDataGedung() {
        System.out.println("Gedung: " + namaGedung);
        ruang1.tampilkanRuang();
        ruang2.tampilkanRuang();
    }
}
```

```
public class DemoComposition {
    public static void main(String[] args) {
        Gedung gedung = new Gedung("Gedung Laboratorium ITERA");
        gedung.tampilkanDataGedung();
    }
}
```

Output:

```
Gedung: Gedung Laboratorium ITERA
Ruang Kelas: Ruang 101
Ruang Kelas: Ruang 102
```

Pada contoh tersebut, object `RuangKelas` dibuat di dalam constructor class `Gedung`. Hal ini menunjukkan composition karena object `RuangKelas` menjadi bagian kuat dari object `Gedung`.



Gambar 7.3 Ilustrasi Composition antara Gedung dan RuangKelas

Gambar ini menunjukkan hubungan composition, yaitu class **Gedung** memiliki object **RuangKelas** sebagai bagian yang kuat. Object **RuangKelas** dibuat dan dikelola oleh object **Gedung**, sehingga keberadaannya bergantung pada object **Gedung**.

7.7 Perbedaan Association, Aggregation, dan Composition

Association, aggregation, dan composition sama-sama menjelaskan hubungan antar object. Namun, ketiganya memiliki tingkat kekuatan hubungan yang berbeda.

Aspek	Association	Aggregation	Composition
Makna hubungan	Menggunakan atau berhubungan	Memiliki secara lemah	Memiliki secara kuat
Ketergantungan object	Rendah	Sedang	Tinggi
Object bagian bisa berdiri sendiri	Ya	Ya	Tidak atau sangat bergantung
Contoh	Mahasiswa mengambil MataKuliah	Kelas memiliki Mahasiswa	Gedung memiliki RuangKelas
Simbol UML	Garis biasa	Diamond kosong	Diamond hitam
Istilah umum	uses-a	has-a lemah	has-a kuat

Secara sederhana, association menunjukkan hubungan umum. Aggregation menunjukkan hubungan kepemilikan yang tidak terlalu kuat. Composition menunjukkan hubungan kepemilikan yang sangat kuat.

7.8 Implementasi Association dalam Java

Pada association, object biasanya digunakan sebagai parameter method atau atribut yang tidak menunjukkan kepemilikan kuat. Contohnya adalah mahasiswa mengambil mata kuliah.

```
class MataKuliah {
    String kode;
    String nama;

    MataKuliah(String kode, String nama) {
        this.kode = kode;
        this.nama = nama;
    }
}
```

```
class Mahasiswa {
    String nama;
    String nim;

    Mahasiswa(String nama, String nim) {
        this.nama = nama;
        this.nim = nim;
    }

    void ambilMataKuliah(MataKuliah mk) {
        System.out.println("Nama Mahasiswa : " + nama);
        System.out.println("NIM          : " + nim);
        System.out.println("Mengambil MK      : " + mk.kode + " - " +
mk.nama);
    }
}
```

```
public class PraktikAssociation {
    public static void main(String[] args) {
        Mahasiswa mhs = new Mahasiswa("Dewi", "123140001");
        MataKuliah mk = new MataKuliah("IF221", "Pemrograman
Berorientasi Objek");

        mhs.ambilMataKuliah(mk);
    }
}
```

Output:

```
Nama Mahasiswa : Dewi
NIM           : 123140001
Mengambil MK  : IF221 - Pemrograman Berorientasi Objek
```

Pada program tersebut, object `MataKuliah` dikirim ke method `ambilMataKuliah()`. Object Mahasiswa menggunakan object `MataKuliah`, tetapi tidak memilikinya secara kuat.

7.9 Implementasi Aggregation dalam Java

Pada aggregation, satu object memiliki kumpulan object lain, tetapi object yang dimiliki dibuat di luar class pemilik. Ini berarti object bagian masih bisa ada secara mandiri.

```
class Mahasiswa {
    String nama;
    String nim;

    Mahasiswa(String nama, String nim) {
        this.nama = nama;
        this.nim = nim;
    }

    void tampilkanData() {
        System.out.println(nim + " - " + nama);
    }
}
```

```
class KelasPraktikum {
    String namaKelas;
    Mahasiswa[] mahasiswa;

    KelasPraktikum(String namaKelas, Mahasiswa[] mahasiswa) {
        this.namaKelas = namaKelas;
        this.mahasiswa = mahasiswa;
    }

    void tampilkanKelas() {
        System.out.println("Kelas Praktikum: " + namaKelas);
        System.out.println("Daftar Mahasiswa:");

        for (Mahasiswa m : mahasiswa) {
            m.tampilkanData();
        }
    }
}
```

```
public class PraktikAggregation {
    public static void main(String[] args) {
        Mahasiswa m1 = new Mahasiswa("Andi", "123140001");
        Mahasiswa m2 = new Mahasiswa("Budi", "123140002");
        Mahasiswa m3 = new Mahasiswa("Citra", "123140003");

        Mahasiswa[] daftarMahasiswa = {m1, m2, m3};

        KelasPraktikum kelas = new KelasPraktikum("PBO-A",
daftarMahasiswa);
        kelas.tampilkanKelas();
    }
}
```

Output:

Kelas Praktikum: PBO-A
 Daftar Mahasiswa:
 123140001 - Andi
 123140002 - Budi
 123140003 - Citra

Relasi ini termasuk aggregation karena object Mahasiswa dibuat terlebih dahulu, lalu dimasukkan ke object KelasPraktikum. Mahasiswa tetap dapat digunakan di tempat lain meskipun object kelas tidak digunakan.

7.10 Implementasi Composition dalam Java

Pada composition, object bagian dibuat di dalam object utama. Ini menunjukkan hubungan kepemilikan yang kuat.

```
class JadwalPraktikum {
    String hari;
    String jam;

    JadwalPraktikum(String hari, String jam) {
        this.hari = hari;
        this.jam = jam;
    }

    void tampilkanJadwal() {
        System.out.println("Hari: " + hari);
        System.out.println("Jam : " + jam);
    }
}
```

```
class KelasPraktikum {
    String namaKelas;
    JadwalPraktikum jadwal;

    KelasPraktikum(String namaKelas) {
        this.namaKelas = namaKelas;

        // Object JadwalPraktikum dibuat di dalam KelasPraktikum
        jadwal = new JadwalPraktikum("Senin", "08.00 - 10.00");
    }

    void tampilkanInfoKelas() {
        System.out.println("Kelas Praktikum: " + namaKelas);
        jadwal.tampilkanJadwal();
    }
}
```

```
public class PraktikComposition {
    public static void main(String[] args) {
        KelasPraktikum kelas = new KelasPraktikum("PBO-B");
        kelas.tampilkanInfoKelas();
    }
}
```

Output:

Kelas Praktikum: PBO-B
 Hari: Senin
 Jam : 08.00 - 10.00

Pada contoh tersebut, object `JadwalPraktikum` dibuat di dalam class `KelasPraktikum`. Artinya, jadwal menjadi bagian dari kelas praktikum dan dikelola langsung oleh class tersebut.

7.11 Studi Kasus Praktikum: Sistem Kelas Mahasiswa ITERA

Pada studi kasus ini, mahasiswa akan membuat program sederhana yang menggabungkan beberapa relasi antar object. Studi kasus menggunakan tema kelas praktikum agar dekat dengan kegiatan mahasiswa.

Program ini menggunakan:

- association antara Mahasiswa dan MataKuliah
- aggregation antara KelasPraktikum dan Mahasiswa
- composition antara KelasPraktikum dan JadwalPraktikum

Source Code

```
class Mahasiswa {
    String nama;
    String nim;

    Mahasiswa(String nama, String nim) {
        this.nama = nama;
        this.nim = nim;
    }

    void ambilMataKuliah(MataKuliah mk) {
        System.out.println(nama + " mengambil mata kuliah " + mk.nama);
    }

    void tampilkanData() {
        System.out.println(nim + " - " + nama);
    }
}

class MataKuliah {
    String kode;
    String nama;

    MataKuliah(String kode, String nama) {
        this.kode = kode;
        this.nama = nama;
    }

    void tampilkanMataKuliah() {
        System.out.println("Mata Kuliah: " + kode + " - " + nama);
    }
}
```

```

class JadwalPraktikum {
    String hari;
    String jam;

    JadwalPraktikum(String hari, String jam) {
        this.hari = hari;
        this.jam = jam;
    }

    void tampilkanJadwal() {
        System.out.println("Hari Praktikum : " + hari);
        System.out.println("Jam Praktikum : " + jam);
    }
}

```

```

class KelasPraktikum {
    String namaKelas;
    MataKuliah mataKuliah;
    Mahasiswa[] daftarMahasiswa;
    JadwalPraktikum jadwal;

    KelasPraktikum(String namaKelas, MataKuliah mataKuliah, Mahasiswa[]
daftarMahasiswa) {
        this.namaKelas = namaKelas;
        this.mataKuliah = mataKuliah;
        this.daftarMahasiswa = daftarMahasiswa;

        // Composition: jadwal dibuat di dalam class KelasPraktikum
        this.jadwal = new JadwalPraktikum("Rabu", "13.00 - 15.00");
    }

    void tampilkanInfoKelas() {
        System.out.println("Nama Kelas: " + namaKelas);
        mataKuliah.tampilkanMataKuliah();
        jadwal.tampilkanJadwal();

        System.out.println("Daftar Mahasiswa:");
        for (Mahasiswa m : daftarMahasiswa) {
            m.tampilkanData();
        }
    }
}

```

```

public class SistemKelasITERA {
    public static void main(String[] args) {
        MataKuliah mk = new MataKuliah("IF221", "Pemrograman
Berorientasi Objek");

        Mahasiswa m1 = new Mahasiswa("Andi", "123140001");
        Mahasiswa m2 = new Mahasiswa("Budi", "123140002");
        Mahasiswa m3 = new Mahasiswa("Citra", "123140003");

        m1.ambilMataKuliah(mk);
        m2.ambilMataKuliah(mk);
        m3.ambilMataKuliah(mk);

        System.out.println();

        Mahasiswa[] daftar = {m1, m2, m3};

        KelasPraktikum kelas = new KelasPraktikum("PBO IF-01", mk,
daftar);
        kelas.tampilkanInfoKelas();
    }
}

```

Output

Andi mengambil mata kuliah Pemrograman Berorientasi Objek
 Budi mengambil mata kuliah Pemrograman Berorientasi Objek
 Citra mengambil mata kuliah Pemrograman Berorientasi Objek

Nama Kelas: PBO IF-01
 Mata Kuliah: IF221 - Pemrograman Berorientasi Objek
 Hari Praktikum : Rabu
 Jam Praktikum : 13.00 - 15.00
 Daftar Mahasiswa:
 123140001 - Andi
 123140002 - Budi
 123140003 - Citra

Pada studi kasus tersebut, beberapa relasi muncul dalam satu program. Relasi Mahasiswa dan MataKuliah termasuk association karena mahasiswa menggunakan object mata kuliah. Relasi KelasPraktikum dan Mahasiswa termasuk aggregation karena mahasiswa dibuat di luar kelas, lalu dimasukkan ke dalam kelas praktikum. Relasi KelasPraktikum dan JadwalPraktikum termasuk composition karena jadwal dibuat di dalam class KelasPraktikum.

7.12 Kesalahan Umum dalam Relasi Antar Object

Saat mempelajari relasi antar object, mahasiswa sering mengalami kesulitan membedakan jenis relasi. Hal ini wajar karena association, aggregation, dan composition memang terlihat mirip pada awalnya.

Beberapa kesalahan umum yang sering terjadi adalah:

1. Semua relasi dianggap inheritance

Tidak semua hubungan antar class harus menggunakan inheritance. Jika hubungannya “memiliki” atau “menggunakan”, maka lebih tepat memakai relasi antar object.

2. **Salah membedakan aggregation dan composition**
Aggregation bersifat kepemilikan lemah, sedangkan composition bersifat kepemilikan kuat.
3. **Object selalu dibuat di class utama tanpa alasan jelas**
Dalam desain OOP, perlu dipikirkan object mana yang seharusnya membuat, menyimpan, atau menggunakan object lain.
4. **Tidak memahami hubungan has-a dan uses-a**
Hubungan has-a biasanya menunjukkan kepemilikan, sedangkan uses-a menunjukkan penggunaan.
5. **Terlalu banyak class tetapi relasinya tidak jelas**
Class yang banyak tidak selalu berarti desain program baik. Yang penting adalah hubungan antar class harus masuk akal.

7.13 Tugas Percobaan

Percobaan 1: Association

Buat program yang menggambarkan hubungan antara Mahasiswa dan MataKuliah.

Ketentuan:

1. Buat class Mahasiswa.
2. Buat class MataKuliah.
3. Buat method `ambilMataKuliah()` pada class Mahasiswa.
4. Object MataKuliah digunakan sebagai parameter method.
5. Tampilkan nama mahasiswa dan mata kuliah yang diambil.

Percobaan 2: Aggregation

Buat program yang menggambarkan hubungan antara KelasPraktikum dan Mahasiswa.

Ketentuan:

1. Buat class Mahasiswa.
2. Buat class KelasPraktikum.
3. Object Mahasiswa dibuat di luar class KelasPraktikum.
4. Class KelasPraktikum menyimpan array object Mahasiswa.
5. Tampilkan daftar mahasiswa dalam kelas praktikum.

Percobaan 3: Composition

Buat program yang menggambarkan hubungan antara Gedung dan RuangKelas.

Ketentuan:

1. Buat class RuangKelas.
2. Buat class Gedung.
3. Object RuangKelas dibuat di dalam class Gedung.
4. Tampilkan nama gedung dan daftar ruang kelas.

5. Jelaskan mengapa relasi tersebut termasuk composition.

Percobaan 4: Studi Kasus Mini

Buat program **Sistem Kelas Praktikum ITERA** dengan ketentuan berikut:

1. Memiliki class Mahasiswa.
2. Memiliki class MataKuliah.
3. Memiliki class JadwalPraktikum.
4. Memiliki class KelasPraktikum.
5. Gunakan association antara Mahasiswa dan MataKuliah.
6. Gunakan aggregation antara KelasPraktikum dan Mahasiswa.
7. Gunakan composition antara KelasPraktikum dan JadwalPraktikum.
8. Tampilkan informasi kelas praktikum secara lengkap.

Pertanyaan Diskusi

Jawablah pertanyaan berikut sebagai bagian dari pemahaman konsep.

1. Apa yang dimaksud dengan relasi antar object?
2. Apa perbedaan association dan aggregation?
3. Apa perbedaan aggregation dan composition?
4. Mengapa tidak semua hubungan antar class harus menggunakan inheritance?
5. Apa arti hubungan **has-a** dalam OOP?
6. Apa arti hubungan **uses-a** dalam OOP?
7. Mengapa object pada aggregation masih dapat berdiri sendiri?
8. Mengapa composition disebut sebagai relasi kepemilikan yang kuat?

7.14 Tugas Laporan Praktikum

Setelah menyelesaikan seluruh percobaan pada BAB 7, mahasiswa diminta membuat laporan praktikum. Format laporan dibuat konsisten dengan BAB sebelumnya agar mahasiswa terbiasa dengan pola kerja yang sistematis dan rapi.

Struktur laporan:

1. Cover
2. Tujuan praktikum
3. Alat dan bahan
4. Langkah percobaan
5. Source code
6. Hasil output program
7. Analisis program
8. Kesimpulan

Ketentuan isi analisis:

Pada bagian analisis, mahasiswa diminta menjelaskan penerapan relasi antar object pada program yang dibuat. Penjelasan minimal mencakup:

- class apa saja yang dibuat
- relasi apa saja yang digunakan
- class mana yang menggunakan association
- class mana yang menggunakan aggregation
- class mana yang menggunakan composition
- perbedaan association, aggregation, dan composition dalam program
- alasan penggunaan relasi tersebut

Mahasiswa juga perlu menjelaskan bagaimana object saling bekerja sama dalam program. Penjelasan tidak perlu terlalu panjang, tetapi harus menunjukkan bahwa mahasiswa memahami hubungan antar class yang dibuat.

Laporan dikumpulkan dalam bentuk **PDF** sesuai ketentuan dosen atau asisten praktikum.

7.15 Penutup

Pada BAB 7 ini, mahasiswa telah mempelajari relasi antar object dalam pemrograman berorientasi objek. Relasi tersebut meliputi association, aggregation, dan composition.

Association digunakan untuk hubungan umum antar object. Aggregation digunakan untuk hubungan kepemilikan yang lemah, sedangkan composition digunakan untuk hubungan kepemilikan yang kuat.

Dengan memahami relasi antar object, mahasiswa dapat membuat program yang lebih dekat dengan sistem nyata. Pemahaman ini juga penting sebagai bekal sebelum masuk ke BAB 8, yaitu **Error Handling dan Java I/O**